project-corebank 작업 정리

2026-11-10 · DB 계정 분리(테이블 소유자 / 앱 계정), 마이그레이션 분리

백엔드 **github.com/Csj930205/project-corebank** 화면 **github.com/Csj930205/project-corebank-front**

266	96	171 / 374	V13 + R
백엔드 테스트 **+7**	화면 테스트 **변경 없음**	Postman 요청 / 검증 **변경 없음**	DB 마이그레이션 **반복 실행 1개 추가**

증가 폭은 11-06 마감 기준(백엔드 259, 화면 96, Postman 171/374, V13). 백엔드 테스트는 로컬과 GitHub Actions(Linux) 양쪽에서 통과.

1. 오늘 한 일

작업	내용	커밋
DB 계정 분리	서버는 앱 계정으로, 마이그레이션만 테이블 소유자로 접속. 앱 계정은 필요한 테이블 권한만 갖고 DDL 은 못 한다	`be084dc`
앱 계정 최소 권한	원장·이체·감사 기록은 SELECT·INSERT 만, 계좌는 잔액·상태·해지 시각 열만 UPDATE	`be084dc`
테스트 환경도 같은 구성	테스트 컨테이너가 같은 초기화 스크립트로 앱 계정을 만들고, 모든 통합 테스트가 앱 계정으로 돈다	`be084dc`
개발 DB 전환	기존 볼륨에 앱 계정 생성, `.env` 키 이름 변경(값 유지) 후 Postman 회귀 전체 통과	–
마이그레이션 분리	마이그레이션은 별도 실행 진입점(`./gradlew migrate`)이 소유자로. 서버는 DB 버전 확인만 하고 소유자 비밀번호가 필요 없다. 아래 5절	`1085735`

11-05 원장 불변성 작업에서 남긴 한계: 앱 계정이 테이블 소유자라 같은 계정으로 트리거를 끄거나(ALTER TRIGGER) TRUNCATE·DROP 하면 막지 못했다. 트리거는 "앱이 실수로 고치는 것" 은 막지만 "앱 계정을 가진 사람이 마음먹고 지우는 것" 은 못 막았다. 오늘 작업으로 이 구멍을 닫았다.

2. 계정 구성

계정	누가 쓰나	할 수 있는 일
테이블 소유자 `corebank` `DB_OWNER_*`	마이그레이션 프로그램 (`./gradlew migrate` , 5절)	테이블·트리거 생성과 변경, 앱 계정에 권한 부여
앱 계정 `corebank_app` `DB_APP_*`	서버 (평소 요청 처리, 일 마감 배치)	접속(CREATE SESSION)과 소유자가 준 테이블 권한만. 자기 스키마에 객체도 못 만든다

- **계정 만들기:** 앱 계정은 DBA 만 만들 수 있어 `docker/oracle-init/01-create-app-user.sh` 가 만든다. compose 는 DB 를 처음 만들 때(`/container-entrypoint-initdb.d`), 기존 볼륨은 `docker compose exec` 로 한 번 실행. 이미 있으면 아무것도 안 해서 여러 번 실행해도 된다. 이름·비밀번호 글자를 제한해 SQL 이 끼어들지 않게 했다.
- **권한 주기:** 소유자가 Flyway 반복 실행 마이그레이션 `R__app_user_grants.sql` 로 준다. 앱 계정 이름은 Flyway 플레이스홀더 `${app_user}` .
- **테이블 찾기:** 앱 계정은 테이블이 없으므로 접속할 때마다 `ALTER SESSION SET CURRENT_SCHEMA = 소유자` (Hikari `connection-init-sql`). SQL·엔티티에 스키마 이름을 붙이지 않아도 되고, JPA 스키마 검사(validate)도 그대로 통과한다.

3. 앱 계정 권한

테이블	권한	이유
ledger_transaction, ledger_entry, transfer	`SELECT, INSERT`	원장은 추가만. 정정은 역분개로
audit_log	`SELECT, INSERT`	감사 기록은 추가만
account	`SELECT, INSERT, UPDATE(balance, status, closed_at)`	바뀌는 열만. 계좌번호·이름·개설일은 앱도 못 바꾼다. DELETE 없음
idempotency_record, customer, admin_user	`SELECT, INSERT`	한 번 저장하면 바꾸는 기능이 없다
login_attempt, rate_limit	`SELECT, INSERT, UPDATE, DELETE`	횟수를 세고, 성공·해제·정리할 때 지운다
daily_closing	`SELECT, INSERT, UPDATE`	실행 상태(RUNNING → 결과) 변경
daily_closing_total, closing_mismatch	`SELECT, INSERT`	마감 결과는 추가만
BATCH_* (Spring Batch 6개)	`SELECT, INSERT, UPDATE`	잡 실행 기록과 진행 상태. 지우지 않는다
시퀀스 14개	`SELECT`	id 발급

- **권한 목록은 한 파일:** `R__` 파일이 바뀌면 Flyway 가 다시 실행하는데, 먼저 앱 계정에 준 권한을 모두 거두고(테이블·열 권한 모두) 목록대로 다시 준다. 그래서 파일에서 지운 권한은 DB 에서도 사라진다.
- **열 권한에 맞춘 코드:** Hibernate 는 엔티티의 바꿀 수 있는 열을 모두 UPDATE 문에 넣는다. `Account.ownerName` 에 `updatable = false` 를 붙여 UPDATE 문이 허용된 열만 쓰게 했다(계좌번호·개설일·고객 id 는 원래 붙어 있었다).
- **앱이 실제로 쓰는 SQL 기준:** JdbcTemplate 의 UPDATE·DELETE 와 엔티티 변경 메서드를 모두 찾아본 뒤 권한을 정했다.

4. 원장 보호: 이제 세 겹

겹	누구를 막나	막힐 때
권한 (오늘)	앱 계정. 원장 UPDATE·DELETE, 트리거 끄기, TRUNCATE, DROP 모두 권한이 없다	`ORA-41900 / ORA-01031`
트리거 (V11, V12)	권한이 있는 테이블 소유자의 UPDATE·DELETE	`ORA-20001 / ORA-20002`
일 마감 대사	소유자가 트리거를 끄고 바꾼 경우. 막지는 못하지만 다음 날 찾아낸다	불일치 기록

5. 마이그레이션 분리: 서버는 확인만

계정을 나눈 뒤에도 Flyway 가 서버 시작 때 소유자로 접속해서 서버 설정에 소유자 비밀번호가 남아 있었다. 마이그레이션을 서버에서 떼어 냈다.

실행	계정	하는 일
`MigrationApplication` `./gradlew migrate`	테이블 소유자	DB 연결과 Flyway 만 켜고(웹·JPA·보안·배치 없음) 마이그레이션한 뒤 끝난다. 설정은 `migration.properties` 를 따로 읽는다

서버 `./gradlew bootRun`	앱 계정	마이그레이션하지 않는다. 시작할 때 이력(`flyway_schema_history` , 읽기 권한만)으로 DB 가 코드와 같은 버전인지 확인(validate)하고, 적용 안 된 파일이 있으면 시작하지 않는다(`SchemaVersionCheck`)

- **배포:** 같은 jar 로 마이그레이션 → 서버 시작. 마이그레이션은 `-Dloader.main=com.corebank.migration.MigrationApplication` 과 Spring Boot `PropertiesLauncher` 로 실행한다. 소유자 비밀번호는 이 단계에만 준다.
- **서버 컴포넌트 스캔과 분리:** 마이그레이션 설정 클래스에 `@Configuration` 을 붙이지 않고 `@ImportAutoConfiguration` 만 붙였다. 그래서 서버가 이 설정을 읽어 들이지 않고, 테스트가 찾는 `@SpringBootConfiguration` 도 하나로 유지된다.
- **테스트도 배포 순서대로:** 테스트 컨테이너가 뜨면 `MigrationApplication` 으로 먼저 마이그레이션하고, 서버는 소유자 비밀번호 없이 앱 계정으로 띄운다. 마이그레이션 프로그램 자체도 매번 검증된다.
- **접속 주소:** 두 설정이 같은 `DB_URL` (선택, 기본은 compose 의 Oracle)을 쓴다.

> **스키마 이름 대소문자**
>
> 처음 돌렸을 때 서버가 `Validate failed: Schema "corebank" doesn't exist yet` 으로 시작하지 않았다. Flyway 는 스키마 이름의 대소문자를 구분하는데, Oracle 은 따옴표 없이 만든 계정 이름을 `COREBANK` 로 저장한다. `.env` 값을 그대로 넘기지 않고 코드에서 대문자로 바꿔 넘기도록 고쳤다. 스키마가 없으면 만들지 않고 실패한다.

6. 테스트 구성과 확인하다 찾은 것

- **테스트 컨테이너:** `@ServiceConnection` 은 계정을 하나만 연결해서 쓰지 않고, `DynamicPropertyRegistrar` 로 `.env` 와 같은 이름의 값(`DB_OWNER_*` , `DB_APP_*`)을 넣는다. 운영 설정 파일은 테스트용으로 따로 고치지 않았다.
- **소유자 접속 `SchemaOwner` :** 대사 테스트의 원장 조작(트리거 잠시 끄기)과 개설일 바꾸기는 앱 계정으로 할 수 없게 되어, 테스트 전용 소유자 접속으로 옮겼다. `JdbcTemplate` 빈을 하나 더 등록하면 스프링이 앱용 JdbcTemplate 을 만들지 않아 레코드로 감쌌다.
- **불변성 테스트 강화:** 원장·감사 기록 수정·삭제를 앱 계정(권한 오류)과 소유자(트리거 오류) 양쪽으로 확인한다.
- **새 테스트 5개** (`DatabaseAccountSeparationIntegrationTest`): 앱은 앱 계정으로 접속하고 기본 스키마는 소유자 / 앱 계정은 객체 0개·시스템 권한은 CREATE SESSION 하나 / DDL 6종 거부 / 계좌 열 권한 / **소유자의 모든 테이블·시퀀스에 앱 권한이 있다**(새 테이블에 권한을 빠뜨리면 여기서 깨진다).
- **새 테스트 2개** (`SchemaVersionCheckIntegrationTest`): 서버의 Flyway 는 앱 계정으로 소유자 스키마(`COREBANK`)를 확인하고 통과 / 테스트 전용 위치의 적용 안 된 `V9999` 를 더하면 확인이 실패한다.

> **Oracle 23ai 의 권한 오류 코드**
>
> 테스트를 처음 돌렸을 때 6개가 실패했다. 권한이 없는 DML 이 예전의 `ORA-01031 insufficient privileges` 가 아니라 `ORA-41900 missing UPDATE privilege on "COREBANK"."TRANSFER"` 처럼 어떤 권한이 빠졌는지 알려 주는 새 코드로 나왔다(DDL 은 여전히 ORA-01031). 또 DDL 거부는 스프링이 `BadSqlGrammarException` 으로 감싸 메시지에 ORA 코드가 없어, 원인 예외에서 확인하도록 고쳤다.

7. 검증

항목	결과	새로 확인한 것

백엔드 테스트 (Testcontainers Oracle)	266 통과	모든 통합 테스트가 앱 계정 권한만으로 통과 = 권한이 모자라지 않다. 새 테스트 7개
Postman 회귀 (실제 서버, 개발 DB)	171 / 374 통과	앱 계정으로 띄운 서버(마이그레이션 분리 후)에서 전체 API·일 마감·감사 로그 동작
마이그레이션 (개발 DB)	통과	`./gradlew migrate` 와 jar(PropertiesLauncher) 둘 다. 서버 로그는 마이그레이션 없이 "15개 확인" 만
개발 DB 직접 시도	모두 거부	sqlplus 로 앱 계정 접속: ALTER TRIGGER·TRUNCATE(ORA-01031), DELETE audit_log(ORA-41900)
GitHub Actions	통과	두 커밋 모두. Linux 에서 초기화 스크립트·마이그레이션 프로그램 실행 포함

8. 남은 한계와 다음 후보

알려진 한계

- 서버는 소유자 비밀번호를 읽지 않지만, 개발 PC 는 `.env` 하나를 마이그레이션과 같이 써서 파일에는 들어 있다. 운영은 배포 단계에만 준다
- 권한 재부여는 "모두 거두고 다시 주기" 라, 서버가 도는 중에 마이그레이션하면 잠깐 권한 오류가 날 수 있다. 무중단 배포라면 차이만 주고 거두는 방식이 필요
- 마이그레이션을 되돌리는 기능(undo)은 없다. 잘못된 변경은 새 버전 파일로 바로잡는다
- V11 주석의 "운영에서는 나눠야 한다" 문구는 이미 실행된 마이그레이션이라 고치지 않았다(체크섬)
- 그 밖에는 README "알려진 한계"(password spraying, 잠금 악용, 고정 구간, 계좌 검색 인덱스 등) 그대로

다음 후보

- 배포 자동화: CI 에서 jar 빌드 → 마이그레이션 → 서버 교체
- 정정(역분개) 거래와 승인 절차
- IP 단위 로그인 제한
- 계좌 검색: 정렬 선택, 커서 방식, 필요한 인덱스
- 일 마감 분산 잠금(ShedLock), 서버 여러 대 대비
- 리프레시 토큰, 토큰 강제 무효화